

COMP9414 Artificial Intelligence

Tutorial topic: foundation models with ai9414

UNSW Sydney

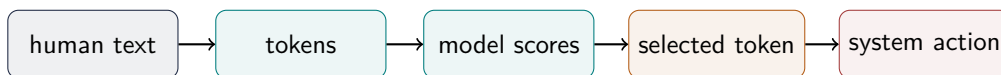
1 Introduction

A foundation model is trained on broad data at scale and can be adapted to a range of downstream tasks. The model is reusable, but it is not a complete system. A useful application may also need task instructions, retrieved evidence, tools, validation, fallback behaviour, and a user or robot interface.

This tutorial practises five ideas from the lecture:

- text is converted into tokens before it reaches a language model;
- a model produces next-token scores, while a decoding rule selects among the candidates;
- prompting, retrieval, tools, workflows, and fine-tuning solve different kinds of problems;
- factual claims and physical actions require grounding, verification, and safe fallback behaviour;
- failures should be diagnosed at the responsible layer of the complete system.

The two local ai9414 explorers make the first two stages inspectable:



Central distinction. The model produces scores from its context. The decoder selects output tokens. The surrounding system supplies evidence, tools, constraints, validation, and consequences. Do not attribute every capability or failure to “the model”.

2 Installation and setup

2.1 Create a Python environment

This tutorial uses ai9414 $\geq$ 0.2.0 and Python 3.11 or newer. On macOS or Linux:

```
mkdir ai9414-foundation-models-tutorial
cd ai9414-foundation-models-tutorial
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "ai9414 $\geq$ 0.2.0"
python -m pip show ai9414
```

On Windows PowerShell:

```
mkdir ai9414-foundation-models-tutorial
cd ai9414-foundation-models-tutorial
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.2.0"
python -m pip show ai9414
```

If you are working inside the course repository, the local environment is already available. Use `uv run ai9414 ...` or `.venv/bin/python -m ai9414 ....`

If an activated environment does not put the `ai9414` command on `PATH`, replace `ai9414` with `python -m ai9414` in any command below.

2.2 Run the explorers

Check the installation and list the curated examples:

```
ai9414 list
ai9414 list --examples foundation-models-tokenisation
ai9414 list --examples foundation-models-decoding
```

Launch the tokenisation explorer:

```
ai9414 demo foundation-models-tokenisation
```

Launch the decoding explorer in a second terminal:

```
ai9414 demo foundation-models-decoding
```

Each command starts a local web app. Leave the terminal command running while using its browser window, then stop it with **Ctrl-C**.

What these examples do not do. The examples do not call a pretrained language model. Tokenisation uses a simplified deterministic byte-pair encoding (BPE) learner, while decoding uses curated scores. This makes the calculations reproducible; entered text does not produce new model scores.

3 Foundation model, decoder, or system?

A foundation model can support many tasks, but it is only one part of an application. Separate the model from the decoding method and the surrounding application system:

Layer	Main responsibility	Example
Foundation model	Produce representations or output scores from context	Assign next-token scores after reading a prompt
Decoder	Choose output tokens or candidate sequences	Greedy selection, sampling, top- k , top- p , beam search
Application system	Supply evidence, tools, constraints, validation, and an interface	Retrieve the current course outline, call a calculator, then cite or abstain

3.1 Task: locate the responsible layer

For each statement, write *model*, *decoder*, *system*, or *more than one layer*. Give one sentence of justification.

1. The next-token distribution assigns 0.35 to **mug**.
2. Only the three highest-probability candidates remain.
3. A current assessment schedule is retrieved.
4. The answer includes a citation that does not support its claim.
5. The same prompt produces different wording on repeated runs.
6. A robot stops because a person entered its workspace.

4 Text becomes tokens

A language model does not receive words directly. A tokeniser maps text to a sequence of discrete units, which can then be mapped to identifiers and embedding vectors.

Mode in the explorer	Rule	Important limitation
Character	Keep every character, including whitespace	Long sequences
Word	Split words and punctuation using a simple rule	Rare words require an ever-growing vocabulary
Learned byte-pair encoding (BPE)	Repeatedly merge frequent adjacent symbols within word-local units	A teaching implementation, not the tokeniser of a named production model

Byte-pair encoding is a family of *subword* tokenisation methods. It starts with small symbols, such as characters, counts adjacent pairs in a training corpus, and repeatedly merges a frequent pair into a larger reusable token. The resulting pieces sit between individual characters and whole words. The lecture introduces this process on the slide “How BPE Builds Subword Tokens”; the explorer lets you replay the same idea step by step.

The teaching BPE learner first splits text into word-local units. Leading whitespace remains attached to the following unit, and merges do not cross unit boundaries. The explorer’s legend shows how whitespace is displayed. Both the corpus and number of merges affect the final token boundaries.

4.1 Task: predict before observing

This task asks you to predict token boundaries before the explorer reveals them. A boundary is a place where one token ends and the next begins. Write a vertical bar | at every predicted boundary.

For example, a simple word-token prediction for **Room A-17** is:

Room | A | - | 17

For each input below, write:

1. **Word prediction:** copy the input and insert | at each expected word-token boundary.

2. **BPE uncertainties:** name two fragments whose boundaries may depend on the learned merges.
3. **Reason:** justify your choices in one sentence.

Use that response format for:

1. Deliver parcel to Office A before 10:30.
2. Send the package to Maximilian and Krzysztof.
3. `for i in range(10): print(i)`

Do not predict character mode: it places a boundary after every character. Your BPE prediction need not match the explorer exactly; identify boundaries that may depend on the corpus.

4.2 Task: compare tokenisation modes

Launch:

```
ai9414 demo foundation-models-tokenisation --example simple_sentence
```

Here, `simple_sentence` is the name of a built-in example, not a filename. The command loads the sentence `Deliver parcel to Office A before 10:30.` into the explorer.

Complete the following steps:

1. Select the character mode and record its token count.
2. Select the word mode and record its token count. Which input characters do not appear as word tokens?
3. Select learned BPE. Before replaying the merges, record the initial token count.
4. Replay every merge and record the final token count.
5. Identify one final token that contains more than one character.
6. Open “Why token count matters”. Explain why the displayed budget is illustrative rather than a real model context limit.

4.3 Task: corpus dependence and rare text

Use these examples:

```
ai9414 demo foundation-models-tokenisation --example rare_names
ai9414 demo foundation-models-tokenisation --example typo_sentence
ai9414 demo foundation-models-tokenisation --example code_example
```

For at least two examples:

1. use the example’s default corpus and replay all 12 merges;
2. switch to a different corpus and replay again;
3. record one fragment that changed;
4. explain why the training corpus changes the learned vocabulary;

5. decide whether fewer tokens necessarily means a better representation.

A useful claim and an unsafe claim. BPE promotes frequent adjacent fragments into reusable tokens. A boundary alone does not establish a token's meaning, and this learner does not reproduce a commercial tokeniser.

5 From scores to a next token

For candidate token i with logit z_i , softmax produces:

$$p_i = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

Temperature $T > 0$ transforms the logits before softmax:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

Lower temperature sharpens a non-uniform distribution; higher temperature flattens it. Temperature changes the distribution. It does not add knowledge, evidence, or reasoning capability.

5.1 Task: work through the lecture distribution

The default decoding example uses this fixed distribution:

Token	mug	cup	bowl	robot	blue	floor	banana	other
Probability	.35	.22	.16	.09	.07	.05	.04	.02

Before opening the explorer, answer:

1. Which token does greedy decoding select?
2. Which tokens remain for $\text{top-}k = 3$?
3. For $\text{top-}p = 0.70$, what is the smallest leading set whose cumulative probability is at least 0.70?
4. What probability mass does that set retain?
5. Renormalise the retained probabilities so that they sum to one.
6. Which methods can produce different selections on repeated runs?

Now launch the matching explorer:

```
ai9414 demo foundation-models-decoding --example lecture_activity
```

Set `top_p` to 0.70 and replay the stages:

1. model scores;
2. temperature transformation;

3. filtering;
4. renormalisation;
5. selection.

Check your retained set and renormalised probabilities against the explorer. Then change to $\text{top-}k = 3$. Explain why $\text{top-}k$ and $\text{top-}p$ happen to retain the same set here even though their rules are different.

5.2 Task: temperature and repeated sampling

Compare:

```
ai9414 demo foundation-models-decoding --example peaked_distribution
ai9414 demo foundation-models-decoding --example flat_distribution
```

For each example:

1. set the selection rule to sampling;
2. compare $T = 0.5$, $T = 1.0$, and $T = 2.0$;
3. record how the largest probability changes;
4. show the repeated-sampling histogram;
5. explain why the histogram is reproducible for a fixed seed but is not identical to the probability distribution after only finitely many draws.

6 From a next token to a sequence

Generation repeats next-token prediction. A locally best token does not necessarily begin the highest-probability complete sequence. This is the same local-versus-global issue encountered in search.

Beam search is a sequence-decoding method introduced in the lecture. At each step it expands the currently retained partial sequences, ranks the new sequences by cumulative score, and keeps only the best B , where B is the *beam width*. Beam width 1 behaves like greedy decoding in this small example; a wider beam can preserve a less likely first token long enough for a stronger complete sequence to emerge.

The sequence-search view in the `lecture_activity` example restricts the first choice to `mug` and `cup`. Their renormalised first-step probabilities are:

$$P(\text{mug}) = \frac{35}{57}, \quad P(\text{cup}) = \frac{22}{57}.$$

Selected second-step probabilities are:

$$P(\text{carefully} \mid \text{mug}) = 0.55, \quad P(\text{safely} \mid \text{cup}) = 0.95.$$

6.1 Task: greedy versus beam search

1. Which first token does greedy decoding choose?
2. Compute $P(\text{mug carefully})$.
3. Compute $P(\text{cup safely})$.
4. Which complete sequence has the larger probability?
5. Open the Sequence Search view and compare your result with the greedy and beam-search cards.
6. Explain, using cumulative probability or cumulative log-probability, why the beam winner can differ from the greedy continuation.
7. Give one cost of retaining a wider beam.

Scope of the sequence tree. The tree is a small teaching example, not a trace from a bundled language model. Beam search is more systematic than greedy search, but it still prunes the full output space and does not guarantee a useful or truthful answer.

7 Choose the right adaptation mechanism

Different requirements belong in different parts of a foundation-model system.

Terminology. *Few-shot prompting* means placing a small number of worked input-output examples before the new input to demonstrate the desired mapping or format. It does not update the model parameters. *Zero-shot prompting* supplies instructions but no worked examples.

Need	Likely mechanism
Clarify the task or output format	Prompt
Demonstrate labels, style, or an input-output mapping	Few-shot examples
Add changing, private, or authoritative knowledge	Retrieval
Obtain exact calculation, current state, or external action	Tool
Enforce a repeatable sequence of operations	Explicit workflow
Change recurring model behaviour when good examples exist	Fine-tuning or an adapter
Acquire a missing core capability	Different model or architecture

7.1 Task: select and justify a mechanism

Choose a primary mechanism for each requirement. Use the sentence form “Use X because the task requires Y ”.

1. Answer questions about a course policy that changes each term.

2. Return one of four fixed incident labels in an exact JSON format.
3. Calculate the examination mark required to reach a course total.
4. Make a frequently repeated specialist mapping more consistent when thousands of reviewed examples are available.
5. Reserve a current tutorial room.
6. Produce three alternative names for a student robotics project.
7. Interpret images when the existing model accepts only text.

Some requirements need more than one mechanism. State any necessary retrieval, validation, or human approval even when the question asks for a primary mechanism.

8 Specify a grounded model interaction

Here, a *model interaction contract* is a written specification of what the model and surrounding application should do. It records evidence, constraints, output format, fallback behaviour, and checks; it is not a legal contract.

TASK	What should be done?
EVIDENCE	Which supplied information may support factual claims?
CONSTRAINTS	What must be preserved, avoided, or handled exactly?
OUTPUT	What form should the result take?
ABSTENTION	When should the system ask, decline, or escalate?
VERIFICATION	How can the result be checked?

8.1 Task: repair an underspecified request

Start with:

Tell me when the assignment is due and what mark I need to pass.

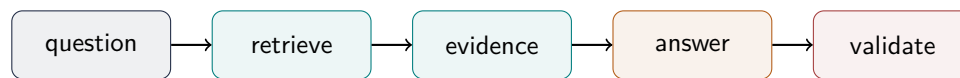
Rewrite it as an interaction contract for a course assistant, separating model instructions from system responsibilities. Your contract must:

- use only the current official assessment schedule for dates and weights;
- obtain the student's marks from an authorised source;
- use a calculator for the required-mark calculation;
- state assumptions and cite evidence identifiers;
- report conflicting sources rather than silently choosing one;
- abstain when an essential value is unavailable.

Then underline which requirements are only instructions to the model and circle which require external system support.

9 Grounding, failure injection, and evaluation

Retrieval-augmented generation can ground an answer in external evidence:



Retrieval can still return a wrong, stale, incomplete, or malicious passage. The generator can ignore good evidence. Evaluation must therefore test the components and the end-to-end system.

What “layer” means here. A layer is a functional subsystem in the application, not a neural-network layer. For the following task, choose the subsystem most directly responsible: *source data and governance*, *retrieval*, *routing or workflow*, *tool*, *model generation*, or *output validation*. If two layers contribute, name a primary layer and then the secondary layer.

9.1 Task: diagnose injected failures

For each failure, fill one row. Use a layer name defined above; state the system’s desired response; and give one observable pass condition.

Injected condition	Likely layer	Desired response	Measure
Last year’s schedule ranks first			
No source contains the exam date			
Two official documents disagree			
A retrieved passage contains instructions for the model			
The correct weights are retrieved but the arithmetic is wrong			
The citation is real but does not support the answer			

Possible measures include retrieval recall, source freshness, tool-routing accuracy, calculation accuracy, citation support, correct abstention, and end-to-end task success.

9.2 Task: design a small evaluation set

Submit eight short plain-text test specifications, one for each category below. Use this three-line format for every answer:

Input/setup: the question and the evidence or system condition
Expected behaviour: what the system should answer or do
Pass condition: the observable result that counts as success

The eight categories are:

1. answer present in one current source;

2. answer absent;
3. ambiguous question;
4. conflicting sources;
5. stale source;
6. exact calculation;
7. irrelevant retrieval;
8. instruction embedded inside evidence.

Include the expected behaviour for every case.

10 Multimodal grounding and action

A vision-language model extends the input or output interface across images and text. A vision-language-action model can make robot actions part of its output interface. Neither change removes the need for sensing, state estimation, planning, control, safety constraints, recovery, or human supervision.

10.1 Task: observability audit

For a single RGB image of a cluttered table, classify each question as:

1. answerable from the image;
 2. requiring external knowledge;
 3. requiring physical interaction or additional sensing;
 4. ambiguous without more context.
- Which objects are red?
 - Which mug belongs to the demonstrator?
 - Which object is heaviest?
 - Which object can this robot grasp?
 - Is the bowl empty?
 - Is the glass hot?

For each answer, name the evidence or sensor that would make the claim defensible.

10.2 Task: diagnose an embodied system

The instruction is:

Put the apple in the bowl.

Classify the main failure layer for each variation as *language*, *perception*, *knowledge*, *planning*, *control*, *safety*, or *alignment*. Then specify a response such as ask, re-observe, retrieve, replan, recover, stop, refuse, or escalate.

1. There are two matching red apples.
2. The bowl is upside down.
3. The apple is behind a glass barrier.
4. The bowl contains hot liquid.
5. A person's hand enters the workspace.
6. The camera moved after calibration.
7. The grasp slips during execution.
8. The instruction conflicts with a laboratory safety rule.

Stakes change with the output interface. A wrong text answer can misinform a user. A wrong action can cause physical harm. A robot system therefore needs explicit stopping, constraint checking, recovery, and escalation behaviour outside the generative model.

11 Your mission

Complete these tasks during the tutorial:

1. **Tokenisation.** Predict token boundaries, compare the three modes, and investigate corpus dependence.
2. **Decoding.** Calculate and verify filtering, temperature effects, repeated sampling, and greedy versus beam search.
3. **Adaptation and grounding.** Choose mechanisms and write the course-assistant interaction contract.
4. **Evaluation.** Diagnose the injected failures and design the eight-case evaluation set.
5. **Multimodal systems.** Complete the observability and embodied failure exercises.

12 Task checklist

If this was a test or exam, you would be asked to submit:

- the environment check result;
- a tokenisation worksheet with predictions, counts, and the corpus comparison;
- a decoding worksheet with filtering, temperature, sampling, and beam search results;
- your mechanism choices and model interaction contract;
- the failure diagnosis table and evaluation cases;
- your multimodal and embodied-system classifications.

A Formula quick reference

These formulas support the decoding and sequence-search tasks. In order, they convert logits to probabilities, apply temperature, calculate a sequence probability, and repeat the sequence calculation in log space. Log space turns products into sums and reduces numerical underflow.

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

$$P(t_1, \dots, t_n) = \prod_{i=1}^n P(t_i \mid t_1, \dots, t_{i-1})$$

$$\log P(t_1, \dots, t_n) = \sum_{i=1}^n \log P(t_i \mid t_1, \dots, t_{i-1})$$

B Decoding rule quick reference

Rule	Operation	Repeatable?
Greedy	Select the largest probability	Yes, absent ties or other changes
Sampling	Draw according to the probability distribution	Not normally
Top- k	Retain exactly the leading k candidates, then renormalise	The set is repeatable; sampling need not be
Top- p	Retain the smallest leading set whose cumulative mass reaches p , then renormalise	The set is repeatable; sampling need not be
Beam search	Retain several partial sequences by cumulative score	Yes for fixed scores and tie-breaking

C System-design card

For any proposed foundation-model application, answer:

1. What outcome is required?
2. Where should factual information come from?
3. What interpretation or generation should the model perform?
4. What evidence must retrieval select?
5. What requires exact state, calculation, or external action?
6. What output structure and supporting evidence are required?
7. When should the system ask, abstain, stop, recover, or escalate?
8. Which ordinary and failure-oriented cases will be evaluated?

D Demo command quick reference

```
ai9414 list
ai9414 list --examples foundation-models-tokenisation
ai9414 list --examples foundation-models-decoding

ai9414 demo foundation-models-tokenisation
ai9414 demo foundation-models-tokenisation --example rare_names
ai9414 demo foundation-models-tokenisation --example code_example

ai9414 demo foundation-models-decoding
ai9414 demo foundation-models-decoding --example peaked_distribution
ai9414 demo foundation-models-decoding --example flat_distribution
```

E Python quick reference

```
from ai9414.foundation_models import TokenisationExplorer

app = TokenisationExplorer(example="rare_names")
app.set_options(tokeniser_mode="bpe",
                 corpus="names_and_codes", num_merges=12)
app.show()

from ai9414.foundation_models import DecodingExplorer

app = DecodingExplorer(example="lecture_activity")
app.set_options(selection_rule="top_p", temperature=1.0,
                 top_p=0.70, seed=9414)
app.show()
```

References

- [1] Rishi Bommasani and others. *On the Opportunities and Risks of Foundation Models*. Stanford Center for Research on Foundation Models, 2021. <https://arxiv.org/abs/2108.07258>.
- [2] Ashish Vaswani and others. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017. <https://arxiv.org/abs/1706.03762>.
- [3] Alec Radford and others. *Learning Transferable Visual Models From Natural Language Supervision*. Proceedings of ICML, 2021. <https://arxiv.org/abs/2103.00020>.
- [4] Anthony Brohan and others. *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. 2023. <https://arxiv.org/abs/2307.15818>.
- [5] Moo Jin Kim and others. *OpenVLA: An Open-Source Vision-Language-Action Model*. Conference on Robot Learning, 2024. <https://arxiv.org/abs/2406.09246>.
- [6] Oliver Obst. *ai9414: Interactive teaching demos for artificial intelligence*. <https://pypi.org/project/ai9414/>.